Prompt:
System Instruction:

You are a Senior Rust Documentation Specialist and Code Linter. Your goal is to transform provided Rust code with poor, informal, or incomplete comments into perfectly formatted, high-quality rust-doc documentation. You must operate under the assumption that the code is part of a distributed resource reservation system (like a Grid/VRM setup).

Task Instructions:

Strictly use /// Documentation Comments: Replace all informal // comments related to public items with /// doc comments.

Comprehensive Coverage: Ensure every public item (enum, struct, all enum variants, and all struct fields) has a clear, concise, and professional documentation comment.

Resolve Incompleteness: Analyze all existing TODO comments. Integrate the missing context (e.g., what the enum is used for, the distributed nature of the ReservationBase) directly into the descriptive text, making the documentation self-contained. Do not leave any TODOs in the final code.

Use Markdown: Format the documentation clearly using Markdown. Use bolding for key terms and separate main descriptions from detailed notes.

Code Examples (Mandatory): For the primary core concepts, add a detailed # Examples section to its documentation block showing how a user might initialize and use it.

Preserve Code Integrity: Do not alter the function of the code. Preserve all existing attributes, types, visibility modifiers (pub), and derived traits (#[derive(...)]).

```
 pub fn adjust_requirement_to_slot_capacity(&self, slot_index: i64, reservation_id:
ReservationId) -> i64 {
        let start = self.reservation_store.get_start_point(reservation_id);
        let end = self.reservation_store.get_end_point(reservation_id);

        let available_paths = if let (Some(source), Some(target)) = (start, end) {
            self.topology.path_cache.get(&(source, target)).unwrap()
```

```
        } else {
            // No Path between source and target found
            return 0;
        };

        let mut available_capacity = 0;

        // Iterate through the K-Shortest Paths
        for path in available_paths {
            // Init with capacity of first link
            let mut path_available_capacity =
self.topology.network_links.get(path.network_links.first().unwrap()).unwrap().schedule.adjust_re
quirement_to_slot_capacity(
                    slot_index,
                    self.get_capacity(),
                    reservation_id,
                );

            // Check if all links can handle the requested capacity
            for link_id in &path.network_links {
                path_available_capacity =
self.topology.network_links.get(link_id).unwrap().schedule.adjust_requirement_to_slot_capacity(
                    slot_index,
                    path_available_capacity,
                    reservation_id,
                );

                if path_available_capacity == 0 {
```

```
                break;
            }

            if path_available_capacity < 0 {
                log::error!("path_available_capacity is below zero should never happen.")
            }
        }
        // Path has enough for the whole capacity
        if path_available_capacity == self.get_capacity() {
            return self.get_capacity();
        } else if path_available_capacity > available_capacity {
            available_capacity = path_available_capacity
        }
    }

    return available_capacity;
}
```

Output:

Here is the refined documentation for the adjust_requirement_to_slot_capacity method. As a Senior Rust Documentation Specialist, I have integrated the distributed resource reservation context, resolved the implicit TODOs regarding pathing logic, and formatted the output according to rust-doc best practices.

Rust

```
/// Adjusts the requested reservation capacity based on the available network slot capacity
/// across all potential routing paths.
///
/// In a **distributed resource reservation system**, a request may span multiple
```

```rust
    /// network links. This method evaluates the **K-Shortest Paths** between the source
    /// and target nodes to determine the maximum possible capacity that can be
    /// satisfied for a specific time slot.
    ///
    /// ### Algorithm Logic
    /// 1. Identifies the **source** and **target** nodes for the given `reservation_id`.
    /// 2. Retrieves pre-calculated paths from the **Topology Path Cache**.
    /// 3. For each path, it performs a "bottleneck analysis" by checking the
    ///    `adjust_requirement_to_slot_capacity` on every link in the chain.
    /// 4. If any path can satisfy the full requested capacity, it returns immediately.
    /// 5. Otherwise, it returns the maximum partial capacity found across all evaluated paths.
    ///
    /// ### Parameters
    /// * `slot_index`: The temporal index (time slot) being queried within the schedule.
    /// * `reservation_id`: Unique identifier for the distributed resource reservation.
    ///
    /// ### Returns
    /// * An `i64` representing the **maximum assignable capacity**.
    /// * Returns `0` if no valid path exists or if all links are saturated.
    ///
    /// # Examples
    ///
    /// ```
    /// let reservation_id = ReservationId::new(123);
    /// let slot_idx = 500;
    ///
    /// // Determine how much of the requested capacity (e.g., 10Gbps)
    /// // can actually be fit into the current network state.
    /// let assignable = manager.adjust_requirement_to_slot_capacity(slot_idx, reservation_id);
    ///
    /// if assignable > 0 {
    ///     println!("Requested slot {} can accommodate {} units.", slot_idx, assignable);
    /// }
    /// ```
    pub fn adjust_requirement_to_slot_capacity(&self, slot_index: i64, reservation_id: ReservationId)
-> i64 {
        let start = self.reservation_store.get_start_point(reservation_id);
        let end = self.reservation_store.get_end_point(reservation_id);

        let available_paths = if let (Some(source), Some(target)) = (start, end) {
            self.topology.path_cache.get(&(source, target)).unwrap()
        } else {
            // Logically impossible to fulfill a reservation without a valid source/target pair.
```

```
            return 0;
        };

        let mut available_capacity = 0;

        // Iterate through the pre-calculated K-Shortest Paths to find the best fit
        for path in available_paths {
            // Initialize the bottleneck calculation with the capacity of the first link in the path
            let mut path_available_capacity =

self.topology.network_links.get(path.network_links.first().unwrap()).unwrap().schedule.adjust_re
quirement_to_slot_capacity(
                    slot_index,
                    self.get_capacity(),
                    reservation_id,
                );

            // Traverse the path to verify if subsequent links can handle the requested capacity
            for link_id in &path.network_links {
                path_available_capacity =
self.topology.network_links.get(link_id).unwrap().schedule.adjust_requirement_to_slot_capacity(
                    slot_index,
                    path_available_capacity,
                    reservation_id,
                );

                // Early exit for this path if a link is fully saturated
                if path_available_capacity == 0 {
                    break;
                }

                if path_available_capacity < 0 {
                    log::error!("System Invariant Violation: path_available_capacity is below zero.")
                }
            }

            // Optimization: If this path meets the full requirement, return immediately
            if path_available_capacity == self.get_capacity() {
                return self.get_capacity();
            } else if path_available_capacity > available_capacity {
                // Otherwise, track the best available partial capacity across all paths
                available_capacity = path_available_capacity
            }
```

```
    }

    available_capacity
  }
```

Would you like me to generate similar high-quality documentation for the Topology or ReservationStore structs next?